

pqc-hsm-fpga 整体实现讲解

写给 Java 开发者 —— 从零理解一台 FPGA 硬件密码机

- [1. 这是个什么东西](#)
- [2. 两个世界：PS 和 PL](#)
- [3. PL 里有什么](#)
- [4. 随机数是怎么生成的](#)
- [5. 密钥是怎么生成和存储的](#)
 - [5.1 对称密钥（AES / SM4）—— 真正的”不出硬件”](#)
 - [5.2 ML-KEM 私钥 —— 诚实说：它出了硬件](#)
 - [5.3 主密钥层次 —— 在主机软件栈里，而且根是个桩](#)
 - [5.4 一句话总结](#)
- [6. 后量子算法：PS 和 PL 各做哪一块](#)
 - [6.1 ML-KEM（FIPS 203）—— 数学全在 PL](#)
 - [6.2 ML-DSA（FIPS 204，签名）—— 只有零件，没有整机](#)
 - [6.3 对称与国密的分工](#)
- [7. 安全边界怎么执行](#)
 - [怎么证明门控真的有效](#)
 - [安全世界那条路怎么走通的](#)
- [8. 软件栈：两条独立的线（这点容易搞混）](#)
 - [路径 A：真的在 FPGA 上跑的](#)
 - [路径 B：主机软件栈（PKCS#11 那一套）](#)
 - [顺便：PKCS#11 \$\approx\$ JCA Provider](#)
- [9. 完整数据流（一次 ML-KEM 解封装）](#)
- [10. 关键取舍（每条都有代价）](#)
- [11. 老实说：哪些没做到](#)
- [12. 不用板子就能跑的](#)

本文面向没有 FPGA / 硬件 / 密码学背景的软件开发者，尽量用 Java / 服务端的概念做类比。

内容全部依据仓库 `zu3eg-fpga-crypto` 分支的[实际代码](#)写成；凡是项目自己标注为“未做到 / 是桩 / 没接上”的地方，本文一律照实标出，不做美化。

这是一份独立交付的教学文档，尚未收进仓库的 `docs/`。

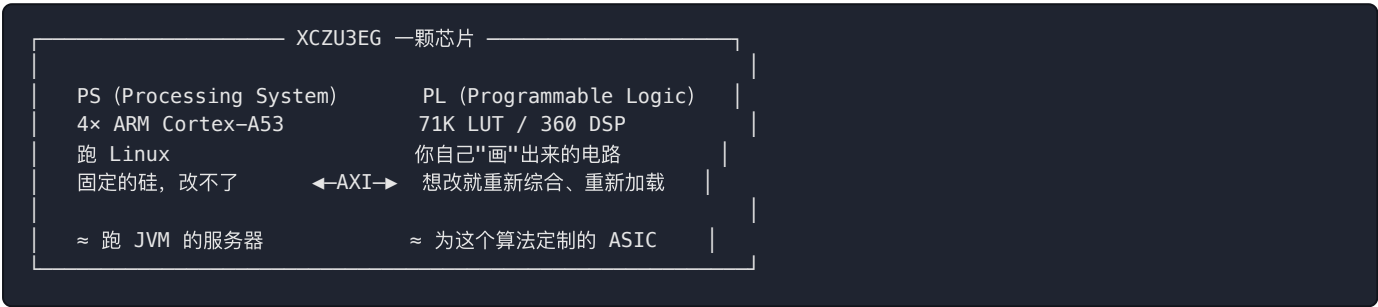
1. 这是个什么东西

一块 FPGA 开发板 (Xilinx XCZU3EG)，被做成了一台**硬件密码机**。

这颗芯片里有两半：一半是 ARM CPU (跑 Linux)，一半是可编程逻辑 (可以”造电路”)。项目把密码算法、随机源、密钥存储、访问控制全部造在了可编程逻辑那一半里，CPU 上的 Linux 只负责下命令、搬数据。

卖点就一句话：密钥和运算在一个软件够不着的地方。不是”软件不去读”，是”读的那条路在电路不存在”。

2. 两个世界：PS 和 PL



- PS 就是你熟悉的那台机器。跑 Linux，有 SD 卡、网口、串口。
 - PL 是空白的可编程逻辑。你用 Verilog 描述一个电路，工具把它”综合”成配置文件 (bitstream，约 35 分钟)，Linux 运行时把这个文件灌进去 (约 2.7 秒)，芯片就变成那个电路了。
 - 两边通过 AXI 总线连接。对软件来说，PL 就是一段物理地址 (0x8000_0000 开始) —— CPU 往那儿写个数，就是在操作 PL 里的电路。
- 关键区别：**PS 上的代码是一条一条执行的；PL 里的电路是**全部同时通电工作**的，每个时钟周期所有信号一起更新。所以 PL 擅长的是”把一个算法摊开成并行电路”，以及”精确控制每一步花多少个周期” (这对常数时间密码学很重要)。
- 本项目 PL 跑在 75 MHz，占用**半块器件** (35,611 LUT / 50.5%，140 DSP，15.5 BRAM)。

3. PL 里有什么

PL 被切成 6 个 64 KB 的地址窗口，每个窗口后面挂一个独立的功能块：

地址	是什么	干什么
0x8000_0000	TRNG	真随机数源 (环形振荡器 + 健康检测 + Keccak 调理)
0x8001_0000	key_vault	对称密钥仓，只进不出
0x8002_0000	sym	AES-128/256、SM4、SM3
0x8003_0000	mlkem	ML-KEM 512/768/1024 的 KeyGen / Encaps / Decaps
0x8004_0000	金丝雀	跟 key_vault 完全相同的一份，只差一个安全参数，存在的意义就是被拒绝

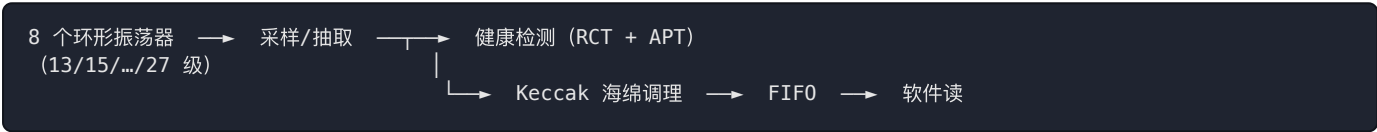
地址	是什么	干什么
0x8005_0000	风扇	温度观测口（不在密码边界内）

每个窗口前面都挡着一道 **AXI 防火墙**，按总线上的安全标记位放行或拒绝。

还有一条**完全不经过总线**的关键连线：**key_vault** 用 256 根私有导线把密钥直接送进 AES/SM4 核。这根线不出芯片，总线上没有它的地址。

4. 随机数是怎么生成的

全部来自 PL，没有一比特是软件 PRNG。



熵从哪来：环形振荡器是奇数个反相器首尾相连，会自己不停振荡。振荡频率由芯片内部的传播延迟决定，而**传播延迟受热噪声影响、逐周期抖动**——这个物理抖动就是全部的熵来源。用一个跟它异步的时钟去采样，采到的比特就带有物理不可预测性。整条链上这是唯一不是数字逻辑的一环。

后面三步都不产生熵，只是保护和整形：

- 健康检测**（SP 800–90B 要求）：RCT 查”同一个值连续出现太多次”（噪声源卡死），APT 查”窗口内某个值占比异常”（分布偏斜）。**一直在跑，不是启动时查一次**。报警就立刻停止输出、清空熵池、重跑启动检测。
- 调理**：用 Keccak 海绵（复用 SHA–3 那个核）而不是自制白化器——因为 SP 800–90B 认可的构造可以直接把输出熵记为 $\min(\text{输出长度}, \text{输入熵})$ ，自创的得自己另行论证。
- FIFO 缓冲**：软件读一次 **RDATA** 弹出一个 32 位字，**读即消耗**。

实测熵：导出 1,048,576 个**调理前**样本，用 SP 800–90B 非 IID 估计器算出 $H = 0.871 \text{ bit/sample}$ 。健康检测的阈值是从这个实测值反推的，不是拍脑袋定的。

谁用它：

- 应用调 **SDFE_GenerateRandom** 直接取
- ML–KEM KeyGen 的种子 **d**、**z**（64 字节）来自它
- ML–KEM Encaps 的消息 **m**（32 字节）来自它

也就是说：每一把在这台机器上生成的后量子密钥，其随机性都来自 PL 里那 8 个环。

5. 密钥是怎么生成和存储的

这里必须分三类讲，因为保证强度完全不同。

5.1 对称密钥（AES / SM4）—— 真正的”不出硬件”

软件写入 → key_vault 的 8 个槽（每槽 256 bit）→ 只能沿私有导线出去进算法核
↑
没有读出通路

- **生成**：可以外部导入（`SDFE_ImportKey`），也可以直接把 TRNG 的输出写进去。
- **存储**：PL 里的寄存器阵列（不是 BRAM）。选寄存器是因为擦除能在一个时钟周期内全清，不存在”擦了一半”的窗口。
- **保护**：RTL 里没有任何一条从密钥寄存器通往总线数据线的导线。软件能做的只有：写入、COMMIT（提交）、LOCK（锁定）、ERASE（擦除）、查询”这个槽装了没”。
- **怎么用**：软件告诉加密核”用第 3 号槽”（一个 3 位的槽号），密钥本体走那 256 根私有线进核。“读密钥”这件事在总线上没有对应的地址，而不是”有地址但被门控了”。

Java 类比：像一个只有 setter 没有 getter 的对象 —— 但不是”getter 是 private”（反射能绕），而是编译出来的字节码里根本没有任何读这个字段并返回的指令。

验证方式也够硬：装 8 把不同的密钥，然后逐字节扫遍整个地址空间，断言这 64 个密钥字一个都不出现；同时用同一把密钥算出的密文与 GB/T 32907 官方向量逐字节一致 —— 密钥确实进了核，只是没有路能读到它。板上和仿真都跑了这个反证。

几条附加规则：

- 只有 COMMIT 之后密钥才可用（写了一半的不算）
- LOCK 之后既不可写也不可擦
- **防拆信号是单向的**：能触发擦除，永远不能反向解除，只有复位能清
- 被拒的操作会留痕（计数器 + 第一次违规的现场）

5.2 ML-KEM 私钥 —— 诚实说：它出了硬件

这是项目自己反复强调、不许含糊的一点。

- **生成**：在 PL 里生成（种子来自 PL 的 TRNG，展开在 PL 的 ML-KEM KeyGen 核里）。
- **但是**：KeyGen 经 AXI 把 `公钥 ek || 私钥 dk` 一起返回给软件。为什么？因为要跟 NIST ACVP 官方测试向量逐字节对拍，必须能拿到 dk。
- **存储**：daemon 把 dk 留在自己的进程内存里，只给应用一个整数句柄。
- **所以**：从应用的视角看，私钥不出接口（应用从头到尾没见过 dk）；但从硬件的视角看，它确实出了硬件。root 可以 dump daemon 的内存拿到它。

项目的原话是：这两句话不一样，不能含糊成一句”私钥不出硬件”。

	对称密钥	ML-KEM 私钥
存在哪	PL 的寄存器	daemon 的进程内存
应用能读到	不能	不能
daemon 能读到	不能	能
root 能读到	不能	能

对称密钥		ML-KEM 私钥
保证的性质	结构性（没有导线）	约定性（软件不暴露）

5.3 主密钥层次 —— 在主机软件栈里，而且根是个桩

这部分不在 FPGA 上，属于另一条线（见第 8 节）。它是一套完整的 HSM 密钥管理模型：

```
KDR（设备绑定的根密钥，32 字节 — 当前是个桩）
└─ KEK      包裹静态密钥材料，不能离开设备
RMK（恢复主密钥，随机生成）
└─ BEK      包裹备份 blob，可以搬到替换设备上
           └─ Shamir M-of-N 分片（GF(256) 上）
```

为什么要两把包裹密钥：目的互相冲突。受设备绑定密钥保护的很安全，但设备一坏就找不回来；受可携带密钥保护的能找回来，但保护强度不超过分片保管的强度。每个槽位带一个策略位决定它参不参与备份 —— 所以设备身份密钥可以被有意做成不可恢复。

密钥库文件长这样：

```
槽 blob := 明文元数据 || AES-256-GCM(KEK, aad=元数据, 密钥材料 || PIN 材料)
文件    := 头 || 槽 blob × N || KMAC256(整文件)
```

元数据只认证不加密 —— 所以不解包就能看到某个槽的算法、用途、策略，同时仍然能发现篡改。写盘是原子的（临时文件 → fsync → rename → fsync 目录）。

每次状态迁移追加一条审计记录，SHA3-256 哈希覆盖前一条 —— 但纯哈希链挡不住把整个文件重写的攻击者，所以链头由 ML-DSA 设备身份密钥签名并锚定在设备之外。

但是 KDR 是个桩。src/crypto/kdr.c 里就是一个固定常量，字面量本身写着 "PQC-HSM STUB KDR -- NOT SECRET!!"。真实设备会从 eFUSE、BBRAM 或 PUF 取它 —— 这块板上 eFUSE 是不可逆的（只有一块板），BBRAM 需要 JTAG。所以设备绑定不是真的。

5.4 一句话总结

- 对称密钥：真的锁在硬件里，这条经得起推敲
- ML-KEM 私钥：只是不出接口，项目自己承认差距
- 主密钥层次：设计完整但根是桩，而且跑在主机软件栈上不在 FPGA 上

6. 后量子算法：PS 和 PL 各做哪一块

这是核心问题。答案分算法看。

6.1 ML-KEM（FIPS 203） —— 数学全在 PL

三个操作各有一个完整的硬件核，PS 不做任何 ML-KEM 数学运算。

算法环节	在哪
种子 d 、 z 、消息 m 的随机源	PL (TRNG)
SHA-3 / SHAKE (G、H、PRF、XOF、J 五种用途)	PL (一个 Keccak 实例服务全部用途)
生成公开矩阵 $\hat{A}$ (拒绝采样)	PL
生成噪声向量 (CBD 中心二项分布)	PL
NTT / 逆 NTT (256 点数论变换)	PL
基乘、Montgomery/Barrett 模约减	PL
压缩 / 解压缩、字节编解码	PL
Decaps 的重加密 + 密文比对 + 隐式拒绝	PL
以上全部的编排 (状态机)	PL
参数集选择、把字节按标准顺序排好	PS
写寄存器、轮询状态、读结果	PS
句柄表、会话管理	PS

PS 那一侧做的事，本质上就是”驱动”——拼输入缓冲、踢一脚、等完成、取结果。一次 Decaps 大约是：

```
写 MODE → 灌 3488 字节输入 → 写 START → 轮询 DONE → 读 32 字节输出
```

核内部三个核的关系有个漂亮的地方：Decaps 需要”重加密”这一步，它直接例化了 Encaps 核，而不是重写一份加密逻辑——因为按 FIPS 203 的定义，这两处运算逐比特相同。少一份实现就少一个可以各错各的地方。

正确性锚定：ML-KEM 512/768/1024 三个参数集的 KeyGen/Encaps/Decaps，在真硅上与 NIST ACVP 官方向量 20/20 逐字节一致。

常数时间：Decaps 的隐式拒绝路径是唯一不容商量的地方。密文比对不许提前退出（否则能从耗时读出第几个字节开始不同，足以逐字节恢复明文），最终选择用位掩码而不是分支。整个核的拍数只由参数集决定，与数据内容无关。实测：合法密文 vs 翻转一比特的密文各 200 次，中位延迟差 0.000%。

6.2 ML-DSA (FIPS 204, 签名) —— 只有零件，没有整机

- PL 里有 13 个算子模块 (NTT、rounding、hint、sample 等)，都已对照参考模型验证过
- 但没有串成完整的 KeyGen / Sign / Verify 核
- 所以这条路上的签名目前跑在软件里 (主机栈用 liboqs)

这是项目明确列出的局限。

6.3 对称与国密的分工

	在哪	说明
AES-128 / AES-256	PL	对照 FIPS 197

	在哪	说明
SM4	PL	对照 GB/T 32907
SM3	PL	对照 GB/T 32905
SM2	没有	国密送检的一块大缺口，项目自己承认

注意：常被一并提起的”SM2/3/4”—— 本项目 SM2 一个门电路都没有。

对称这条路的分工特别干净：软件只说”用哪个算法、用哪个槽的密钥、这是明文分组”，密钥本身从头到尾不经过 PS，也不经过总线。

7. 安全边界怎么执行

三层，从弱到强：

- ① 地址译码完全精确。一个寄存器有且只有一个地址能到它。孔径高位、槽号、槽内偏移高位、32 位对齐 —— 四条判据必须全成立，否则就地拒绝。为什么这么严？因为PS 侧的保护单元根本覆盖不到 PL 这个地址窗口（Xilinx UG1085 有定论），所以 PL 里的这道是唯一的执行点。镜像地址等于给防线开成千上万个等价入口。实测：一个 64 KB 槽位里恰好 64 个地址可达，其余 16,320 个全部拒绝。
- ② AXI 防火墙按安全标记门控。ARM TrustZone 让总线上每一笔事务都自带一个”安全/非安全”标记位（AxPROT[1]），由硬件填，软件伪造不了。防火墙要求这个位是”安全”才放行。而且策略是编译期参数不是运行时寄存器 —— 综合完就固化在硅上，没有任何一条总线路径能改它。
拒绝时返回的是总线错误（DECERR）而不是”成功 + 返回 0” —— 悄悄返回零会让攻击者以为自己读到了东西，拒绝必须是可见的。
- ③ key_vault 没有读出路径。上面 5.1 讲过了。这一层最强，因为它不依赖”每一处都记得检查”。

怎么证明门控真的有效

这是个很聪明的构造。问题是：如果所有核都设成”仅安全世界可访问”，Linux 一个寄存器都读不到 —— 能证明”什么都被拒了”，但证明不了算法是对的。

所以做了两版 bitstream：

第一版：功能核开放（Linux 能直接跑 KAT 验证算法正确），另外在槽 4 放一个跟 key_vault 完全相同、只差一个安全参数的实例 —— 它存在的全部意义就是被拒绝。

板上实测同一个地址、两个世界：

```
地址 0x8004_0000
安全世界读到 0x00010000    ← 成功
普通世界收到 Bus error     ← 被总线拒绝
同时：地址 0x8003_0000（开放的核）
同一个普通世界读到 0x00010000 ← 成功
```

两半都需要 —— 第二半才排除掉”这个地址本来就不可达”。

第二版：把四个功能核全部设成仅安全世界可访问，整套 KAT 改由安全世界驱动。因为第一版留了个缺口 —— 被门控保护的只是金丝雀那个空壳，不是密码核本身。

安全世界那条路怎么走通的

Linux 跑在普通世界，它发的每笔事务都带“非安全”标记。要让访问带上“安全”标记，就得从 EL3（ARM 的最高特权级，Secure Monitor）发出。

链路是：

```
应用 → daemon → 内核模块 → SMC 指令 → EL3 固件 (BL31) → AXI → PL
└─ 跨特权级，root 也改不了对面
```

- 给 ARM Trusted Firmware (BL31) 打补丁，加了两个自定义服务：**读一个 32 位字 / 写一个 32 位字**
- **白名单在 BL31 里判**，不在内核模块里。理由很硬：内核模块 root 能换掉，BL31 在 EL3 普通世界改不了 —— **把策略放在内核模块里等于把锁挂在门外面**
- 白名单是 PL 译码判据的**子集**，绝不更宽。每个槽只放行该核实际存在的寄存器偏移，风扇整核排除
- 三条自我约束：边界全是编译期常量、判据函数无循环无指针算术（一眼能审完）、**不做块传输**（性能不构成把口子开大的理由）

诚实说明：这个 SiP 暴露的是**MMIO 粒度**的读写，操作序列还是在普通世界拼装的。所以**root 读不到密钥材料**，但仍然能**装载和使用 key_vault 的槽位**。要堵上这一点需要**操作粒度**的 SiP（“用句柄 H 解封装这段密文”而不是“往这个地址写这个字”）—— 那意味着一个跑在 EL3 里的带缓冲状态机，而 EL3 没有异常处理，一次取指出错就是一个 CPU 核当场卡死。项目没有尝试。

8. 软件栈：两条独立的线（这点容易搞混）

仓库里有两条**互不相连**的软件路径，必须分清：

路径 A：真的在 FPGA 上跑的

```
应用（不链接任何密码库）
└─ SDF 风格接口 (GM/T 0018 的国密版 PKCS#11)
libsdfe — 无状态，一行密码运算都不做，只翻译成一条请求
└─ UNIX socket
pqchsm_fpgad — 串行化硬件访问、管会话和句柄、把业务操作展开成寄存器序列
└─ ioctl
/dev/secmmio 内核模块 — 只当信使，无策略
└─ SMC
EL3 固件 → AXI → PL 密码核
```

demo 程序**只链接 libsdfe**，**完全不链接任何密码库** —— 所以它打印出的任何正确结果都只可能来自 FPGA。

daemon 这一层为什么不能省：三个核都是“写模式 → 灌输入 → 启动 → 轮询 → 读输出”的**有状态序列**。两个进程交错驱动同一个核，得到的是互相错位但看起来完全合法的结果 —— **不报错、不崩溃，就是算错**。所以必须有一个串行化的单点。

Java 类比: `libsdfc` $\approx$ gRPC 生成的 client stub; `pqchsm_fpgad` $\approx$ 一个单实例服务; `/dev/secmmio` + SMC $\approx$ JNI 跨界, 只不过跨的是特权级。

路径 B: 主机软件栈 (PKCS#11 那一套)

```
pkcs11-tool / Java 应用
  ↓ PKCS#11 v3.2
src/p11/ — 前端 (几乎是纯翻译层)
  ↓
槽位管理器 / 密钥库 / 备份恢复 / 审计链      ← 5.3 那套密钥层次在这里
  ↓
liboqs 软件实现                                ← 后端是软件, 不是 FPGA
```

这条路目前不接 FPGA。项目自己写明: `accel_transport_t` 之上的主机栈没有对着真实可编程逻辑跑过, “把它接起来是另一件事”。模块的型号字符串就是 `pqchsm-sw`。

为什么不合并: 那个栈连着 keystore、PIN、槽位状态机、KEK 包裹一整套, 把它整个搬到板上要一并带起那套状态, 与“证明硬件能被标准接口调用”这件事无关。两边的线上协议形状保持一致 (定长头 + 长度前缀), 将来合并时不用改调用方。

顺便: PKCS#11 $\approx$ JCA Provider

这个类比很准:

- JCA 定义 `Cipher` / `Signature` / `KeyPairGenerator`, Provider 实现它们
- PKCS#11 定义 `C_Encrypt` / `C_Sign` / `C_GenerateKeyPair`, 设备驱动实现它们
- JDK 自带的 `SunPKCS11` 就是两者的桥

不过 demo 里踩了个坑值得知道: JDK 26 的 JCA 本身认识 ML-DSA/ML-KEM, `SunPKCS11` 也能加载这个模块, 但 `SunPKCS11` 的“PKCS#11 机制码 $\rightarrow$ JCA 算法名”映射表还没加入 v3.2 的 `CKM_ML_KEM` / `CKM_ML_DSA` —— 所以 `KeyPairGenerator.getInstance("ML-DSA", p11Provider)` 直接抛 `NoSuchAlgorithmException`。demo 因此绕过 `SunPKCS11`, 用 JDK 22+ 的 Foreign Function & Memory API 直接调 C ABI, 机制码按数值传。零外部依赖, JDK 自带。

9. 完整数据流 (一次 ML-KEM 解封装)

```
应用：SDFE_Decapsulate_MLKEM(会话，句柄=0，密文 1088 字节，输出缓冲)
| 应用手里只有句柄和密文，没有私钥
▼
libsdfc: 打包成 20 字节头 + 1088 字节载荷，写 socket
▼
daemon: 用句柄查出 dk (2400 字节，应用从没见过)
        拼成 dk || c = 3488 字节，按标准定义的顺序
▼
daemon: 写 MODE=Decaps/768 → 灌 3488 字节 → START → 轮询 → 读 32 字节
        (每一笔都是 ioctl → SMC → EL3 白名单 → AXI)
▼
PL 里的 mlkem_decaps 核:
    解密 → 得到候选明文 m'
    用 m' 重新加密一遍 → 得到 c'
    常量时间比对 c' 与 c (不许提前退出)
    无条件预先算好隐式拒绝值 J(z||c)
    用位掩码选: 相等取正常密钥，不等取拒绝值
▼
原路返回 32 字节共享密钥
```

板上实测：Encaps 侧和 Decaps 侧算出的共享密钥一致，`ddc3f00809b1d926...`。

10. 关键取舍（每条都有代价）

决策	换来什么	代价
PL 跑 75 MHz 而不是 150	最慢的核有 45% 时序余量	吞吐减半
NTT 系数放 BRAM 而不是寄存器	省回 40% 的 LUT	每个蝶形多一拍
蝶形切成两级流水	时序收敛	NTT 从 2305 拍变 3457 拍
密钥仓用寄存器而不是 BRAM	擦除一拍全清，无残留窗口	占 2048 个触发器
防火墙 store-and-forward	权限判据和数据落笔能对上	每笔访问多几拍
EL3 只做单字读写不做块传输	口子最小、判据一眼能审完	每字节一次 SMC，性能难看
Decaps 直接例化 Encaps 核	少一份可以各错各的实现	无
防火墙策略用编译期参数	没有任何总线路径能改策略	改策略要重新综合 35 分钟

性能数字：ML-KEM-512 在 75 MHz 下是 924 / 1339 / 1018 ops/s (KeyGen/Encaps/Decaps)。项目自己说明：**这个数字主要由逐字节的软件 AXI 流量主导，不是核本身的速度**，公布它是为了给出各参数集之间 1 : 1.5 : 2.1 的比例，不是作为性能主张。**没有配 DMA**——那是独立的一块工作。

11. 老实说：哪些没做到

项目自己列的，不是本文加的：

- **ML-KEM 私钥会离开硬件**（见 5.2）
- **root 仍然能驱动硬件**（SiP 是 MMIO 粒度不是操作粒度）

- 密钥派生根是个桩，设备绑定不是真的
- 没有 SM2 核
- ML-DSA 只有算子，没串成完整核
- 防拆信号没人驱动 —— RTL 里有这根线，板上没有机壳、没有开关接它
- 功耗与电磁侧信道有意排除在范围外 —— 常数时间只覆盖时序。理由讲得很硬：没有侧信道实验台就无法验证掩码，交付未经验证的掩码比不做掩码更糟
- PKCS#11 前端跑在软件后端上，不在 FPGA 路径上
- 对 liboqs 和 OpenSSL 不作任何主张 —— 主机路径上每个 PQC 操作都跑在 liboqs 里
- PS 侧的 XMPU/XPPU 用不上 —— 由 Xilinx 手册定论，覆盖不到 PL 窗口
- 两件事需要 JTAG：把 PL 固化进黄金启动镜像、基于 BBRAM 的安全启动
- 这是研究原型，未认证、未加固、不可用于生产

12. 不用板子就能跑的

```
python3 -m venv .venv-rtl && ./venv-rtl/bin/pip install cocotb
brew install icarus-verilog verilator
./tools/rtl_sim.sh           # 197 个 cocotb 测试, 用 Python 驱动 Verilog 仿真
./tools/rtl_lint.sh          # 70 个模块, 零告警
```

```
./tools/fetch_vectors.sh && cmake -S . -B build && cmake --build build -j
ctest --test-dir build --output-on-failure
```

可以继续往下钻的方向：PL 里 ML-KEM 核的内部结构 · TRNG 完整链路与 SP 800-90B 的口径 · 安全边界那三层的 RTL 实现 · 主机栈的密钥层次与备份恢复 · RTL 是怎么写的（跟写软件的根本区别）。